Standard Environment Variables

The values of some environment variables are read by Ruby when it first starts. These variables modify the behavior of the interpreter. Some of the environment variables used by Ruby are listed in the following table.

Variable Name	Description
DLN_LIBRARY_PATH	Specifies the search path for dynamically loaded modules.
HOME	Points to user's home directory. This is used when expanding ~ in file and directory names.
LOGDIR	Specifies the fallback pointer to the user's home directory if $HOME is not set. This is used only by chdir.
OPENSSL_CONF	Specifies the location of OpenSSL configuration file.
PATH	The Unix list of places to look for files, Ruby uses it when calling Kernel#system.
RUBYLIB	Specifies an additional search path for Ruby programs ($SAFE must be 0).
RUBYLIB_PREFIX	(Windows only) Mangles the RUBYLIB search path by adding this prefix to each component.
RUBYOPT	Specifies additional command-line options to Ruby; examined after real command-line options are parsed ($SAFE must be 0).
RUBYPATH	With -S option, specifies the search path for Ruby programs (defaults to PATH).
RUBYSHELL	Specifies shell to use when spawning a process under Windows; if not set, will also check SHELL or COMSPEC.

Table 4—Ruby environment variables

Other Ruby tools like Bundler or your Ruby version manager will also add environment variables.

Ruby uses several environment variables to manage its garbage collector during runtime, these variables all start with RUBY_GC, and you can find them in the man page for Ruby by typing man ruby. Similarly, there are a few variables that start with RUBY_THREAD or RUBY_FIBER that control the amount of size allocated for threads and fibers. These variables are generally used to tweak performance for long-running Ruby programs.

Writing to Environment Variables

A Ruby program may write to the ENV object. On most systems, this changes the values of the corresponding environment variables. However, this change is local to the process that makes it and to any subsequently spawned child processes. This inheritance of environment variables is illustrated in the code that follows. A subprocess changes an environment variable, and this change is inherited by a process that it then starts. However, the change is not visible to the original parent. (This just goes to prove that parents never really know what their children are doing.)

```
puts "In parent, term = #{ENV['TERM']}"
fork do
  puts "Start of child 1, term = #{ENV['TERM']}"
  ENV['TERM'] = "ansi"
```